

Advanced Fuzzing and Crash Analysis

© Richard Johnson 2019

rjohnson@fuzzing.io

Day 2

Day 2 - Agenda

- AFL vs Binaries
- WinAFL
- Attack Surface Analysis
- Memory Debuggers
- Dynamic Binary Instrumentation

AFL vs Binaries

AFL vs ELF Binaries

- By default AFL does not target binaries
- We now have multiple options on Linux using QEMU or DYNINST

AFL QEMU

- AFL's QEMU support adds the ability to trace unmodified binaries
- At its core QEMU is a software CPU emulator, the CPU emulator itself is called the Tiny Code Generator (TCG)
- Modifications to the TCG have been made to log branches in an AFL compatible byte map

AFL QEMU

- By default only the first binary loaded (application binary) will be targeted
- Lcamtuf recommends compiling support libraries you want to fuzz statically into the application, however you want to link system libraries dynamically
- Alternatively use `AFL_INST_LIBS=1` to instrument all code

AFL QEMU

- Use the -Q command line switch to make AFL use QEMU mode
- We may need to increase the amount of memory a fuzzed process is allowed to use with the -m <size> switch e.g. -m 1G
- `afl-fuzz -Q -i <in> -o <out> -- /normal/binary @@`

AFL QEMU Exercise

- Try fuzzing a non-instrumented distribution package with AFL such as tiffinfo
- Try fuzzing some closed source commercial binaries such as the Oracle Outside In Technology parsers in /vulndev/target/oit-ix
- Hint: Since OIT are shared libs you will need to pass the AFL_INST_LIBS=1

AFL-DYNINST

- DynInst is a powerful static binary rewriting framework
- Talos VulnDev team has implemented a plugin to modify binaries on disk to include block based code coverage hooks and AFL compatible logging

AFL-DynInst

- DynInst is a powerful static binary rewriting framework
- Talos VulnDev team has implemented a plugin to modify binaries on disk to include block based code coverage hooks and AFL compatible logging

AFL-DynInst

- Alex Nicolich, 2014 (Talos VulnDev)
- Fastest method for targeting dynamically linked ELF binaries
 - It should work on PE files as well but we have outstanding bugs reported
- Using this code coverage plugin we can statically rewrite dynamically linked ELF files back to disk with modifications in the binary
 - This greatly increases performance
 - Is limited by the ability to do call graph recovery so C++ targets may not be suitable for this approach

AFL-DYNINST Exercise

- `$ ./afl-dyninst.sh`
- Usage: `afl-dyninst-dfvxD -i <binary> -o <binary> -l <library> -e <address> -E <address> -s <number> -S <funcname> -m <size>`
- `-i`: input binary
- `-o`: output binary
- `-d`: do not instrument the binary, only supplied libraries
- `-l`: linked library to instrument (repeat for more than one)
- `-r`: runtime library to instrument (path to, repeat for more than one)

AFL-DYNINST Exercise

- -e: entry point address to patch (required for stripped binaries)
- -E: exit point - force exit(0) at this address (repeat for more than one)
- -s: number of initial basic blocks to skip in binary
- -m: minimum size of a basic block to instrument (default: 1)
- -f: try to fix a dyninst bug that leads to crashes (loss of 20%% performance)
- -S: do not instrument this function (repeat for more than one)
- -D: instrument only a simple fork server and also forced exit functions
- -x: experimental performance mode
- -v: verbose output

AFL-DYNINST Exercise

- `./afl-dyninst.sh -i `which tiffinfo` -o tiffinfo.dyninst -l libtiff.so.5 -f`
- Inserting init callback.
- Skipping library: tiffinfo
- Skipping library: tiffinfo.c
- Skipping library: getpid.c
- ...
- Instrumenting module: libtiff.so.5

AFL-DYNINST Exercise

- Skipping library: libAflDyninst.cpp
 - Skipping library: libAflDyninst.so
 - Saving the instrumented binary to tiffinfo.dyninst ...
 - Did a total of 12859 basic block insertions
 - All done! Happy fuzzing!
-
- `$ ls *tiff*`
 - `libtiff.so.5 tiffinfo.dyninst`
-
- `./afl-fuzz-dyninst.sh -mnone -i /vulndev/afl/testcases/images/tiff/ -o /tmp/afl-tiff -- ./tiffinfo.dyninst @@`

AFL-DYNINST Exercise

- Make a new working copy of the OIT XI libraries
- Instrument with afl-dyninst
- Notice the performance differences between QEMU and DYNINST

WinAFL

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer
- Features
 - Its American Fuzzy Lop! For Windows!
 - Windows API port for memory and process creation
 - DynamoRIO based code coverage
 - Filter based on module
 - Block and Edge tracing modes
 - Block tracing by default due to issues with multi-threading
 - Persistent execution mode

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer
- Persistence
 - Multiple inputs can be parsed without exiting the process
 - DynamoRIO allows hooking of target function
 - User specifies address and number of arguments
 - On function exit, WinAFL repopulates args and loops function
 - User specifies number of loops before process restart

WinAFL

- Ivan Fratric July 2016
 - First performant windows evolutionary fuzzer

- Persistence is key
 - Restart process each time (disable persistence) ~2.3 exec/s
 - Persist 100 iterations before restart ~72 exec/s
 - Persist 1000 iterations ~123 exec/s
 - Persist 10000 iterations ~133 exec/s

```
WinAFL 1.09 based on AFL 2.43b (test_gdiplus.exe)
+- process timing -----+
|   run time : 0 days, 0 hrs, 0 min, 34 sec   |
|   last new path : 0 days, 0 hrs, 0 min, 5 sec |
| last uniq crash : none seen yet             |
| last uniq hang : none seen yet             |
+- cycle progress -----+
| now processing : 0 (0.00%)                  |
| paths timed out : 0 (0.00%)                |
+- stage progress -----+
| now trying : bitflip 1\1                   |
| stage execs : 592/5040 (11.75%)            |
| total execs : 1055                         |
| exec speed : 46.05/sec (slow!)             |
+- fuzzing strategy yields -----+
| bit flips : 0/0, 0/0, 0/0                 |
| byte flips : 0/0, 0/0, 0/0                |
| arithmetics : 0/0, 0/0, 0/0               |
| known ints : 0/0, 0/0, 0/0                |
| dictionary : 0/0, 0/0, 0/0                |
|   havoc : 0/0, 0/0                       |
|   trim : 0.00%/302, n/a                   |
+- overall results -----+
|   cycles done : 0                         |
|   total paths : 16                        |
|   uniq crashes : 0                       |
|   uniq hangs : 0                         |
+- map coverage -----+
|   map density : 3.67% / 4.64%             |
|   count coverage : 1.12 bits/tuple        |
+- findings in depth -----+
| favored paths : 1 (6.25%)                 |
| new edges on : 16 (100.00%)               |
| total crashes : 0 (0 unique)              |
| total tmouts : 0 (0 unique)              |
+- path geometry -----+
|   levels : 2                             |
|   pending : 16                           |
|   pend fav : 1                           |
|   own finds : 15                         |
|   imported : n/a                         |
|   stability : 78.54%                     |
+-----+
```

WinAFL

- WinAFL can be retrieved from Ivan's GitHub:
 - <https://github.com/ivanfratric/winafl>
- WinAFL includes a pre-built binary release that you can use in the `winafl\bin64` directory
- If you have a Windows build environment, you can build custom harnesses
 - VSCode and WindowsSDK compilers should be sufficient

WinAFL

- Usage

-covtype

the type of coverage being recorded: bb (basic block, default) or edge

-coverage_module

module for which to record coverage. Multiple module are supported.

-target_module

module which contains the target function to be fuzzed.

Either -target_method or -target_offset need to be specified together with this option.

WinAFL

- Usage

-target_method

name of the method to fuzz in persistent mode. For this to work either the method needs to be exported or the symbols for target_module need to be available. Otherwise use -target_offset instead.

-target_offset

offset of the method to fuzz from the start of the module.

WinAFL

- Usage

-fuzz_iterations

Maximum number of iterations for the target function to run before restarting the target process.

-nargs

Number of arguments the fuzzed method takes. This is used to save/restore the arguments between runs.

WinAFL

- Usage

-call_convention

The default calling convention is cdecl on 32-bit x86 platforms and Microsoft x64 for Visual Studio 64-bit applications. Possible values:

- * fastcall: fastcall
- * ms64: Microsoft x64 (Visual Studio)
- * stdcall: cdecl or stdcall
- * thiscall: thiscall

WinAFL

- Usage

-debug

Debug mode. Does not try to connect to the server. Outputs a log file containing loaded modules, opened files and coverage information.

-thread_coverage

If set, WinAFL will only collect coverage from a thread that executed the target function

WinAFL

- When using DynamoRIO as our tracing engine, we need to pay very close attention to our harness design
- The default harness shipped with WinAFL is not optimized
- Let's see how we can optimize it with a little refactoring...

WinAFL Exercise: GdiPlus

- WinAFL ships with a simple demonstration that uses GdiPlus.dll APIs to fuzz images
- First we will get WinAFL up and running by executing the DynamoRIO plugin directly and analyzing the log file
- `cd \winafl\bin64`
- `\DynamoRIO\bin64\drrun.exe -c winafl.dll -debug -target_module test_gdiplus.exe -target_offset 0x1270 -fuzz_iterations 10 -nargs 2 -- test_gdiplus.exe \winafl\testcases\images\bmp\not_kitty.bmp`

WinAFL Exercise: GdiPlus

- A log file has been created, if you examine it, you will see we have not specified a module to collect code coverage from, lets do that now
- ```
\DynamoRIO\bin64\dr-run.exe -c winafl.dll -debug -
target_module test_gdiplus.exe -target_offset 0x1270 -
coverage_module gdiplus.dll -coverage_module
WindowsCodecs.dll -fuzz_iterations 10 -nargs 2 --
test_gdiplus.exe
\winafl\testcases\images\bmp\not_kitty.bmp
```
- The new log will now indicate these modules are being traced for coverage

# WinAFL Exercise: GdiPlus

- Now that we've verified our DynamoRIO plugin configuration, we can launch afl-fuzz.exe
- `afl-fuzz.exe -i \winafl\testcases\images\bmp -o out -D \dynamorio\bin64 -t 20000 -- -coverage_module gdiplus.dll -coverage_module WindowsCodecs.dll -fuzz_iterations 5000 -target_module test_gdiplus.exe -target_offset 0x1270 -nargs 2 -- test_gdiplus.exe @@`
- We should now be up and running.. but we can optimize this harness for more speed!

# WinAFL Exercise: GdiPlus

- Copy `\winafl\gdiplus.cpp` to the `bin64` directory as `gdiplus_refactor.cpp`
  - No, really, rename the file.. The compiler will get confused during linking since `gdiplus` is also the name of the library
- Refactor the harness:
  - Create a new function `fuzzit(wchar_t *path)`
  - Only put critical code in `fuzzit()`
  - Export `fuzzit`: `__declspec(dllexport)`
  - Compile: `C:\winafl\bin64>cl /Zi gdiplus_refactor.cpp /link gdiplus.lib`



# WinAFL Exercise: GdiPlus

- Update winafl command line to specify target\_method and nargs
- Result: ~2x faster? Neat!

# WinAFL Exercise: MSXML6

- Now lets do it again, this time from a MSDN example for MSXML6
- Snag the initial harness here:
  - [https://docs.microsoft.com/en-us/previous-versions/windows/desktop/ms754517\(v=vs.85\)](https://docs.microsoft.com/en-us/previous-versions/windows/desktop/ms754517(v=vs.85))
- This one is a bit of a mess, comment out offending lines of code with extreme prejudice!
- No training wheels: refactor into a fuzzable harness

# WinAFL Exercise: Cabinet.dll

- For our next exercise, we will hook directly into a binary that ships with Windows.
- Our target of choice is going to be the Cabinet compression library.
- The first thing we need to do is enumerate binaries in Windows that import our target library so we can select a suitable target.

# WinAFL Exercise: Cabinet.dll

- We can use the dumpbin.exe program from within a Visual Studio command prompt to dump imported library names:

```
C:\Windows\System32>dumpbin /imports notepad.exe
Microsoft (R) COFF/PE Dumper Version 14.16.27026.1
Copyright (C) Microsoft Corporation. All rights reserved.
```

Dump of file notepad.exe

File Type: EXECUTABLE IMAGE

Section contains the following imports:

-snip-

# WinAFL Exercise: Cabinet.dll

- We can use the dumpbin.exe program from within a Visual Studio command prompt to dump imported library names:

```
C:\Windows\System32>dumpbin /imports notepad.exe
```

```
-snip-
```

```
ADVAPI32.dll
```

```
14001A650 Import Address Table
```

```
14001F690 Import Name Table
```

```
0 time date stamp
```

```
0 Index of first forwarder reference
```

```
215 OpenProcessToken
```

```
170 GetTokenInformation
```

```
-snip-
```

# WinAFL Exercise: Cabinet.dll

- We can script this tool to find all binaries that import our target library of cabinet.dll using WSL bash:

```
for i in *.exe; do ["`dumpbin.exe /imports $i | grep -i cabinet`" != ""] && echo $i ; done
```

certutil.exe

cmdl32.exe

DTUHandler.exe

-snip-

- Run this command now to identify candidate binaries for fuzzing .cab files

# WinAFL Exercise: Cabinet.dll

- There are two reasonable candidates for fuzzing Cabinet.dll
- These import Functions by Ordinal .. Lets look at MSDN to see what function we actually want to target
  - <https://docs.microsoft.com/en-us/windows/desktop/devnotes/cabinet-api-functions>
- We need to open our target executable in a debugger or disassembler to determine the callstack that calls our import from Cabinet.dll

# WinAFL Exercise: Cabinet.dll

- First we are looking for the imports in our target binary for FDIcopy
- Next we want to look at the call graph to identify a function entry point that takes a filename and has a call to FDIcopy below
- You might notice there's a pesky MessageBox call in there too
- First get the target running under WinAFL, then go back and patch the MessageBox call out of the binary



# WinAFL Exercise: Foxit PDF Converter

- Download foxit-fuzz.zip, this contains the Foxit PDF Reader and a broken harness for the image import functionality
- I developed this harness for the latest version of foxit based upon a ZDI blog which will give you most of the clues you need but will required additional reversing to complete
  - <https://www.thezdi.com/blog/2017/10/17/wrapping-the-converter-within-foxit-reader>
- We will need to hook the exported API to observe the offsets to the class methods and then extract the new typedefs

# WinAFL Exercise: Foxit PDF Converter

- `set AFL_ROOT=c:\winafl`
- `set INPUT_DIR=c:\winafl\testcases\images\bmp`
- `set DynamoRIO_ROOT=c:\dynamorio`
  
- `cd %AFL_ROOT%\bin32`
- `%DynamoRIO_ROOT%\bin32\drun.exe -c winafl.dll -debug -  
coverage_module FIXME.dll -target_module foxit-fuzz.exe -  
target_method wmain -fuzz_iterations 10 -nargs 2 -- foxit-fuzz.exe  
%INPUT_DIR%\not_kitty.bmp`

# WinAFL Exercise: Foxit PDF Converter

- `afl-fuzz.exe -i %INPUT_DIR% -o foxit_out -D %DynamoRIO_ROOT%\bin32 -t 20000 -- -coverage_module FIXME.dll -target_module foxit-fuzz.exe -target_method wmain -nargs 2 -fuzz_iterations 5000 -- foxit-fuzz.exe @@`
- `afl-fuzz.exe -i %INPUT_DIR% -o foxit_out -D %DynamoRIO_ROOT%\bin32 -t 20000 -- -coverage_module FIXME.dll -target_module foxit-fuzz.exe -target_method convert_to_pdf -nargs 2 -fuzz_iterations 5000 -- foxit-fuzz.exe @@`
- `afl-fuzz.exe -i %INPUT_DIR% -o foxit_out -D %DynamoRIO_ROOT%\bin32 -t 20000 -- -coverage_module FIXME.dll -target_module foxit-fuzz.exe -target_method convert_to_pdf -nargs 2 -fuzz_iterations 5000 -- foxit-fuzz.exe @@ NUL`

# WinAFL Exercise: Hangul

- I helped a previous student develop a custom harness for the Hangul Word Processor's Security module
- Hangul Word Processor supports a OLE Structured Storage just like old MSWORD
- OLE Structured Storage is a single-file filesystem that contains LZ compressed data streams with the objects representing the rich text and layout information

# WinAFL Exercise: Hangul

- This is not a suitable format for fuzzing so we will extract the internal data streams and hook deeper into the program and pass it the decompressed stream data directly!
- First get the `hnc_basic_harness.cpp` running under WinAFL
- Then open our target binary to locate functions that take internal streams
- Next use HncDumper to extract streams

# WinAFL Exercise: Hangul

- Now we have streams extracted, but we need to call into the internal functions that parse them
- Locate the function you want to hook and add the typedef to the harness
- Now call into that function by specifying it's address relative to the library load address

# Attack Surface Analysis

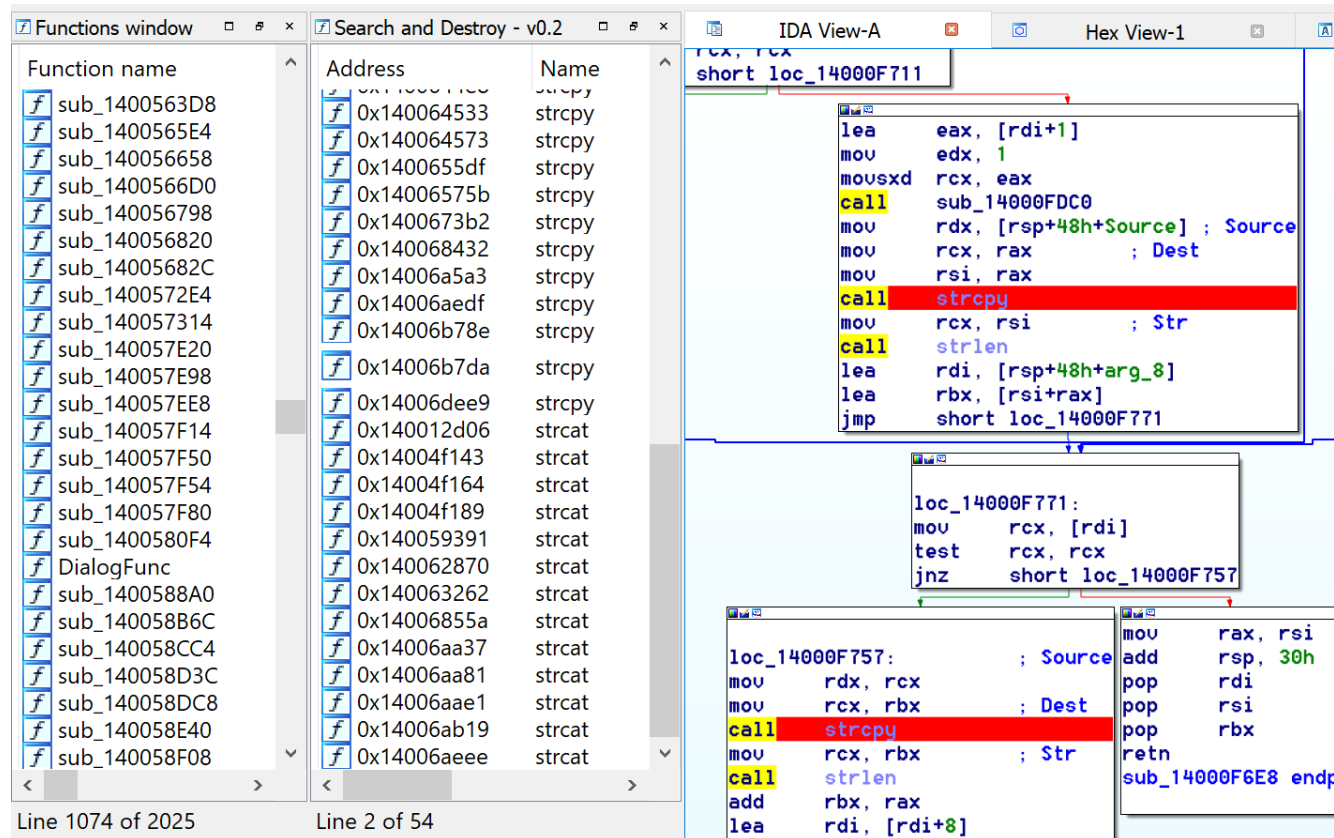
# Attack Surface Analysis

- Identify functions handling externally controlled input
  - read, fread, MapViewOfSection, etc
- Perform a reachability analysis of calls to “interesting functions”
  - <https://github.com/intel/safestringlib/wiki/SDL-List-of-Banned-Functions>
- IDA can help us here



# Search\_and\_Destroy.py

- A simple plugin to highlight API usage in IDA



# highlight\_calls.py

```
COLOR = 0xFF9900 # orange
def highlight_calls(color=COLOR):
 for ea in idautils.Heads():
 if idaapi.isCode(idaapi.getFlags(ea)) and idaapi.is_call_insn(ea):
 current_color = idaapi.get_item_color(ea)
 if current_color == COLOR:
 idaapi.set_item_color(ea, idc.DEFCOLOR)
 elif current_color == idc.DEFCOLOR:
 idaapi.set_item_color(ea, COLOR)

highlight_calls()
```

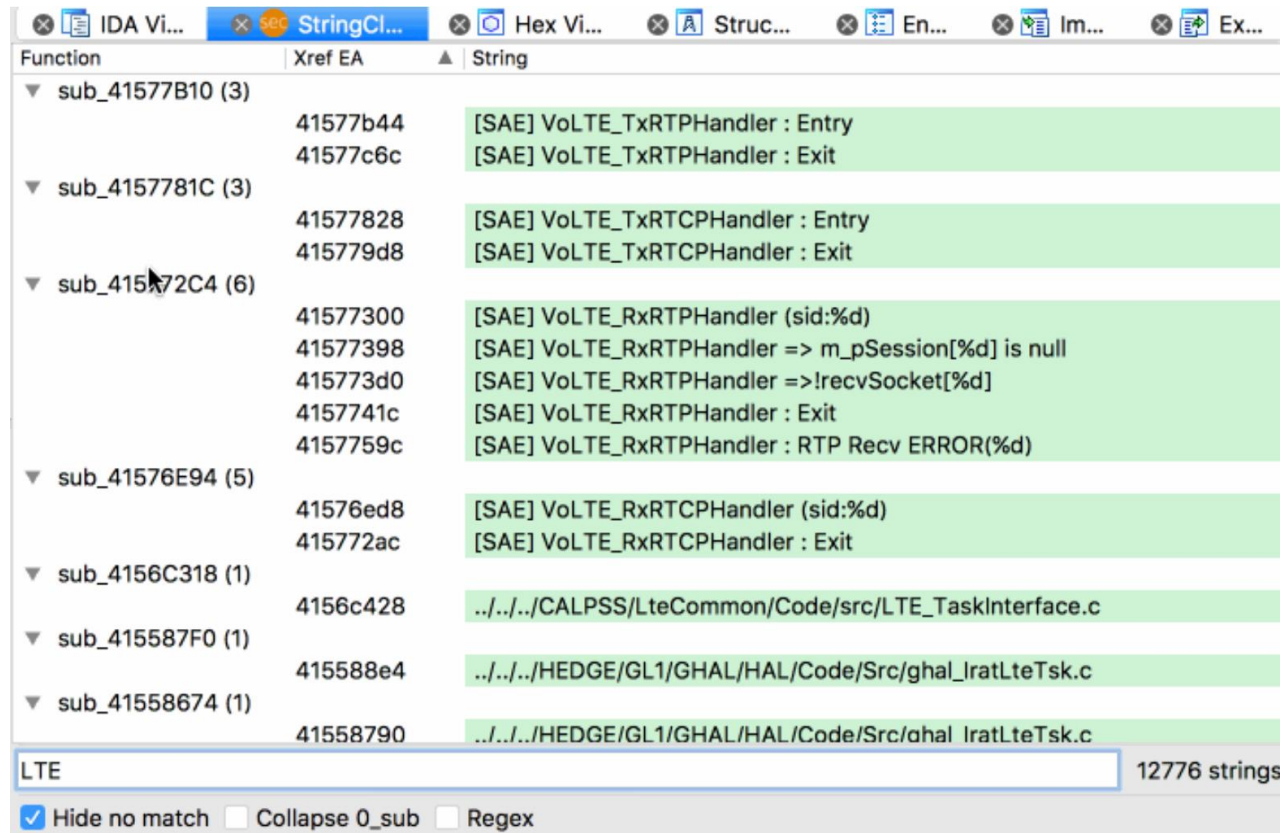
# funcap.py

- Use IDA Debugger to execute your program and collect cross reference and string data among other things
- Use from the output window:

```
import funcap
d.hookSeg() or d.hookFunc(func="myfunc")
d.graph() to view a custom function call tree
d.saveGraph("path")
d.loadGraph("path")
```

# strcluster.py

- Cluster strings by function reference for quicker navigation



# autorename\_functions.py

- This plugin uses several passes of analysis to give functions names
  - Only renames functions that have not been renamed
- Finds unique strings referenced by a function
  - Prefixed with z\_
- Markov model of code and data xrefs in a function
  - Compares xref count with outgoing edges to weight naming priority

# autorename\_functions.py

sub\_140068D94 -> z\_unable\_to\_create\_named\_pipe: : prob 1.000. 1 edges, 1 xrefs, 6 transitions  
sub\_14006CBF0 -> z\_PGP\_fingerprints: : prob 1.000. 1 edges, 1 xrefs, 2 transitions  
sub\_140044400 -> z\_PuTTY\_downstream\_no\_longer\_available: : prob 1.000. 1 edges, 1 xrefs, 7 transitions  
sub\_140004404 -> z\_Host\_Name\_or\_IP\_address: : prob 0.500. 1 edges, 2 xrefs, 2 transitions  
sub\_14003AC0C -> z\_ssh\_rsa: : prob 0.167. 1 edges, 6 xrefs, 6 transitions  
sub\_1400128AC -> z\_Proxy\_error\_Unexpected\_proxy\_error0: : prob 0.500. 1 edges, 2 xrefs, 21 transitions  
sub\_140058F58 -> z\_Error: : prob 0.333. 1 edges, 3 xrefs, 2 transitions  
sub\_140026C1C -> z\_Sent\_environment\_variables: : prob 1.000. 1 edges, 1 xrefs, 6 transitions  
sub\_1400244B0 -> z\_Port\_open\_failed: : prob 0.500. 1 edges, 2 xrefs, 16 transitions  
sub\_14006E428 -> z\_cannot\_do\_XDM\_AUTHORIZATION\_1\_without\_remote\_address\_data: : prob 1.000. 1 edges, 1 xrefs, 11 transitions  
sub\_140062208 -> z\_No\_sessions: : prob 1.000. 1 edges, 1 xrefs, 1 transitions  
sub\_140019C50 -> z\_Agent\_forwarding\_refused0: : prob 0.500. 1 edges, 2 xrefs, 17 transitions

# IFL – Interactive Functions List

- Quickly search for functions that may be a good starting point
- Search by substring and get a quick list of callers

| IFL - Interactive Functions List          |           |                                            |                 |                      |                |           |           |
|-------------------------------------------|-----------|--------------------------------------------|-----------------|----------------------|----------------|-----------|-----------|
| SEARCH: <input type="text" value="read"/> |           |                                            |                 |                      |                |           |           |
| Start                                     | End       | Name                                       | Type            | Args                 | Is referred by | Refers to | Imported? |
| 140081d58                                 | 140081dbe | replace_current_thread_locale_nolock       | void __fastcall | (__acrt_ptd * co...  | 2              | 3         | -         |
| 1400812f4                                 | 1400812f8 | common_refill_and_read_nolock<char>        |                 | (__crt_stdio_stre... | 2              | 0         | -         |
| 14008114c                                 | 1400812f1 | common_refill_and_read_nolock<char>        |                 | (__crt_stdio_stre... | 1              | 9         | -         |
| 14008073c                                 | 140080761 | message_box_wait_thread<wchar_t>           | DWORD __stdcall | (void * const)       | 1              | 1         | -         |
| 14007f850                                 | 14007f8b8 | _configthreadlocale                        | int __cdecl     | (int Flag)           | 1              | 4         | -         |
| 14007e860                                 | 14007e8a3 | _register_thread_local_exe_atexit_callback |                 | (void)               | 1              | 5         | -         |
| 14007e19c                                 | 14007e242 | fread_s                                    | size_t __cdecl  | (void *DstBuf, si... | 1              | 6         | -         |
| 14007e17c                                 | 14007e198 | fread                                      | size_t __cdecl  | (void *DstBuf, si... | 2              | 1         | -         |
| 14007defc                                 | 14007e17b | _fread_nolock_s                            | size_t __cdecl  | (void *DstBuf, si... | 1              | 13        | -         |

size\_t \_\_cdecl **fread** (void \*DstBuf, size\_t ElementSize, size\_t Count, FILE \*File)

| Is referred by 2: |  | Refers to 1: |  |
|-------------------|--|--------------|--|
| Foreign Val.      |  | From Address |  |
| sub_14003B1B8     |  | 14003b22f    |  |
| sub_14003AD1C     |  | 14003ae1e    |  |

# xrefs\_trees.py

- Once you have identified interesting functions the next thing is to explore the trees of calls leading to it
- These are the functions you want to get to while fuzzing or reversing
- xrefs\_trees will give you a text view of xrefs that you can click to navigate



# xrefs\_trees.py

=====

## Cross References To

=====

+ fread()

| + sub\_14003AD1C()

| | + sub\_14003ACDC()

| | | + sub\_14001A344()

| | | | + sub\_140022D88()

| | | | + sub\_14002723C()

| | | | + sub\_140028450()

| | | + sub\_14001C0D8()

| | | | + sub\_14001EEC0()

| | | | | + sub\_140018654()

| | | | | + sub\_140019C50()

| | | | | | + sub\_140022D88() ... [2]

# Exercise

- Finding the entry point to post-auth functionality in `putty.exe`
  - Record coverage with `drcov` up until the password prompt
  - Record again, this time enter the password
- Load traces into Lighthouse and identify the basic blocks that only occur in post-authentication execution

# Memory Debuggers

# Heap Debugging

- Windows: Page Heap
- Linux: efence -> libduma -> libdislocator
- Valgrind -> Address Sanitizer
  
- Targets with source code should use Address Sanitizer
- Linux binary targets can use libdislocator with the QEMU DynInst tracing engines
- Windows binary targets can use PageHeap but its better to integrate into the tracing plugin...

# Page Heap

- Page Heap, Electric Fence, Duma, and Dislocator all work based on memory page permissions
  1. Requested allocations are rounded up to page size and an additional `page_size` is added to the request
  2. The allocator returns the buffer to the interceptor function
  3. The extra page added to the end is marked `NOACCESS`
  4. The buffer is moved to the end of the rounded up page size
  5. Any access beyond the end of the buffer will cause an exception
- We can implement this in a DBI plugin

# Valgrind

- Valgrind is a runtime memory error detector for Linux
- It can detect many vulnerability types
  - Use after free (dangling pointer dereference)
  - Heap buffer overflow
  - Stack buffer overflow
  - Global buffer overflow
  - Use after return
  - Use after scope
  - Initialization order bugs
  - Memory leaks

# Valgrind

```
id:000000,sig:06,src:000143+000093,op:splice,rep:16
==88710== Memcheck, a memory error detector
==88710== Copyright (C) 2002-2015, and GNU GPL'd, by Julian Seward et al.
==88710== Using Valgrind-3.11.0 and LibVEX; rerun with -h for copyright info
==88710== Command: ../qemu-img convert -f vpc -O vmi in /dev/null
==88710==
==88710== Syscall param set_robust_list(head) points to uninitialised byte(s)
==88710== at 0x51A6AF: __pthread_initialize_minimal (nptl-init.c:384)
==88710== by 0x51B3EF: (below main) (in /home/rjohnson/qemu-img)
==88710== Address 0x4000d70 is in the brk data segment 0x4000000-0x40013bf
==88710==
==88710== Conditional jump or move depends on uninitialised value(s)
==88710== at 0x59E4A7: __register_atfork (in /home/rjohnson/qemu-img)
==88710== by 0x59E3C8: __libc_pthread_init (in /home/rjohnson/qemu-img)
==88710== by 0x51A8D2: __pthread_initialize_minimal (nptl-init.c:494)
==88710== by 0x51B3EF: (below main) (in /home/rjohnson/qemu-img)
```

# Address Sanitizer

- AddressSanitizer (ASan) is a memory error detector for C/C++
- It can detect many vulnerability types
  - Use after free (dangling pointer dereference)
  - Heap buffer overflow
  - Stack buffer overflow
  - Global buffer overflow
  - Use after return
  - Use after scope
  - Initialization order bugs
  - Memory leaks
- The average slowdown of the program is ~2x



# Address Sanitizer

```

==3428==ERROR: AddressSanitizer: heap-buffer-overflow on address 0xb6203b80 at pc 0x8087622 bp 0xbfabde58
sp 0xbfabde44
WRITE of size 997 at 0xb6203b80 thread T0
#0 0x8087621 in write_hostent(void*, __sanitizer::__sanitizer_hostent*) /home/rvlfly/Downloads/llvm/projects/compiler-rt/lib/sanitizer_common/sanitizer_common_interceptors.inc:1164
#1 0x80873ec in __interceptor_gethostbyname /home/rvlfly/Downloads/llvm/projects/compiler-rt/lib/sanitizer_common/sanitizer_common_interceptors.inc:1187
#2 0x80d5061 in printHost /home/rvlfly/Projects/Toys/C/GHOST2.c:15
#3 0x80d526b in main /home/rvlfly/Projects/Toys/C/GHOST2.c:50
#4 0x24cbf5 (/lib/tls/i686/cmov/libc.so.6+0x16bf5)

0xb6203b80 is located 0 bytes to the right of 1024-byte region [0xb6203780,0xb6203b80)
allocated by thread T0 here:
#0 0x80bc9c5 in __interceptor_malloc /home/rvlfly/Downloads/llvm/projects/compiler-rt/lib/asan/asan_malloc_linux.cc:74
#1 0x322295 (/lib/tls/i686/cmov/libc.so.6+0xec295)
#2 0x80873dd in __interceptor_gethostbyname /home/rvlfly/Downloads/llvm/projects/compiler-rt/lib/sanitizer_common/sanitizer_common_interceptors.inc:1186
#3 0x80d526b in main /home/rvlfly/Projects/Toys/C/GHOST2.c:50
#4 0x24cbf5 (/lib/tls/i686/cmov/libc.so.6+0x16bf5)
#5 0x80d4fb4 in _start (/home/rvlfly/Projects/Toys/C/a.out+0x80d4fb4)

SUMMARY: AddressSanitizer: heap-buffer-overflow /home/rvlfly/Downloads/llvm/projects/compiler-rt/lib/sanitizer_common/sanitizer_common_interceptors.inc:1164 write_hostent(void*, __sanitizer::__sanitizer_hostent*)
Shadow bytes around the buggy address:
 0x36c40720: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x36c40730: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x36c40740: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x36c40750: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x36c40760: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x36c40770: [fa]fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x36c40780: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x36c40790: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x36c407a0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x36c407b0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
 0x36c407c0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow bytes before the buggy address:
0x36c40710: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40700: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c406a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40690: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40680: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40670: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40660: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40650: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40640: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40630: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40620: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40610: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40600: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c405a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40590: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40580: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40570: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40560: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40550: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40540: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40530: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40520: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40510: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40500: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c404a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40490: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40480: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40470: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40460: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40450: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40440: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40430: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40420: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40410: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c40400: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c403f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c403e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x36c403
```

# Dr. Memory

- Dr. Memory is a memory error checker for binaries
- It works similarly to Valgrind or Address Sanitizer
- Hooks memory access and

# Dr. Memory

```
$ drmemory -- ./dump-prog ./out/crashes/id_000040,sig_11,src_001021,op_flip1,pos_70
~~Dr.M~~ Dr. Memory version 1.11.1
~~Dr.M~~ (Uninitialized read checking is not yet supported for 64-bit)
~~Dr.M~~
~~Dr.M~~ Error #1: UNADDRESSABLE ACCESS: reading 0x0000000000000008-0x000000000000000c 4 byte(s)
~~Dr.M~~ # 0 peparse::getImports [vulndev/corpus/tob-pe-parse-output/pe-parse/parser-library/parse.cpp:1183]
~~Dr.M~~ # 1 peparse::splitBuffer [vulndev/corpus/tob-pe-parse-output/pe-parse/parser-library/buffer.cpp:254]
~~Dr.M~~ # 2 peparse::ParsePEFromFile [/usr/include/c++/4.8/ext/new_allocator.h:110]
~~Dr.M~~ # 3 ld-linux-x86-64.so.2!_dl_fixup [../sysdeps/x86_64/dl-irel.h:32]
~~Dr.M~~ # 4 __libc_csu_init [vulndev/corpus/tob-pe-parse-output/pe-parse/parser-library/buffer.cpp:305]
~~Dr.M~~ # 5 libc.so.6!__libc_start_main [/build/eglibc-SvCtMH/eglibc-2.19/csu/libc-start.c:287]
~~Dr.M~~ # 6 _start [/usr/include/c++/4.8/iostream:74]
~~Dr.M~~ Note: @0:00:00.993 in thread 120862
~~Dr.M~~ Note: instruction: mov 0x08(%rax) -> %esi
~~Dr.M~~
~~Dr.M~~ ERRORS FOUND:
~~Dr.M~~ 1 unique, 1 total unaddressable access(es)
```

# Exercise

- Observe any crashes you have generated under the various memory checkers and become familiar with the output of each
- Run your non-crashing fuzzed corpus under your preferred memory checker, did it find any crashes AFL did not?

# Lighthouse

- Markus Gaasedelen, 2017
- Code coverage visualizer for IDA
- Inputs DynamoRIO drcov format
- Uses binary map for very fast diffing and merging of traces

# Lighthouse

IDA - C:\Users\user\Desktop\LighthouseDev\testcase\boombox.i64 (boombox.exe)

File Edit Jump Search View Debugger Options Windows Help

Functions window

Function name

- sub\_1400012B0
- sub\_1400016F0
- sub\_140001930
- sub\_140001B20
- sub\_140001D10
- sub\_140001ED0
- sub\_140002060
- sub\_140002160
- sub\_140002260
- sub\_140002420
- sub\_140002590
- sub\_140002700
- sub\_1400027C0
- sub\_1400028D0
- sub\_1400029D0
- sub\_140002A60
- sub\_140002AF0
- sub\_140002B50
- sub\_140002BF0
- sub\_140003280
- sub\_140003320
- sub\_140003820
- main
- SetConsoleTextAttribute
- \_security\_check\_cookie
- pre\_c\_init
- pre\_cpp\_init
- NtCurrentTeb

IDA View-A, Coverage Overview

Hex View-1

Strings window

Structures

Enums

Imports

Exports

Coverage Overview

| Coverage % | Function Name | Address     | Blocks Hit | Instructions Hit | Function Size |
|------------|---------------|-------------|------------|------------------|---------------|
| 0.00%      | sub_140001000 | 0x140001000 | 0 / 1      | 0 / 6            | 32            |
| 0.00%      | sub_140001030 | 0x140001030 | 0 / 5      | 0 / 12           | 35            |
| 69.57%     | sub_140001060 | 0x140001060 | 5 / 10     | 32 / 46          | 149           |
| 0.00%      | sub_140001100 | 0x140001100 | 0 / 5      | 0 / 17           | 72            |
| 24.00%     | sub_140001150 | 0x140001150 | 2 / 3      | 6 / 25           | 136           |
| 0.00%      | sub_140001250 | 0x140001250 | 0 / 5      | 0 / 23           | 81            |
| 72.56%     | sub_1400012B0 | 0x1400012B0 | 13 / 20    | 156 / 215        | 1074          |
| 0.00%      | sub_1400016F0 | 0x1400016F0 | 0 / 16     | 0 / 122          | 562           |
| 60.22%     | sub_140001930 | 0x140001930 | 5 / 10     | 56 / 93          | 483           |
| 71.15%     | sub_140001B20 | 0x140001B20 | 12 / 16    | 74 / 104         | 495           |
| 43.52%     | sub_140001D10 | 0x140001D10 | 10 / 16    | 47 / 108         | 446           |
| 21.88%     | sub_140001ED0 | 0x140001ED0 | 6 / 15     | 21 / 96          | 393           |
| 40.74%     | sub_140002060 | 0x140002060 | 5 / 8      | 22 / 54          | 248           |
| 39.62%     | sub_140002160 | 0x140002160 | 5 / 8      | 21 / 53          | 242           |
| 67.37%     | sub_140002260 | 0x140002260 | 10 / 17    | 64 / 95          | 436           |
| 0.00%      | sub_140002420 | 0x140002420 | 0 / 11     | 0 / 79           | 359           |
| 100.00%    | sub_140002590 | 0x140002590 | 6 / 6      | 62 / 62          | 322           |
| 0.00%      | sub_140002700 | 0x140002700 | 0 / 9      | 0 / 53           | 187           |
| 78.05%     | sub_1400027C0 | 0x1400027C0 | 14 / 19    | 64 / 82          | 266           |
| 35.29%     | sub_1400028D0 | 0x1400028D0 | 5 / 18     | 24 / 68          | 247           |
| 53.85%     | sub_1400029D0 | 0x1400029D0 | 4 / 9      | 14 / 26          | 87            |
| 53.85%     | sub_140002A60 | 0x140002A60 | 4 / 9      | 14 / 26          | 87            |
| 0.00%      | sub_140002AF0 | 0x140002AF0 | 0 / 1      | 0 / 17           | 81            |
| 100.00%    | sub_140002B50 | 0x140002B50 | 1 / 1      | 31 / 31          | 145           |

Composer

C - 33.54% Hide 0% Coverage

Line 2 of 77 11.47% (-380,-) (191,1) 00000000 00000000 (Synchr

AU: idle | Down | Disk: 16GB

# Exercise

- Explore the coverage generated by the earlier file fuzzing
- Use lighthouse to batch load the coverage of your generated corpus
- Load any crashing inputs individually and cross reference with Valgrind / GDB output

# Dynamic Binary Instrumentation



# Dynamic Binary Instrumentation

- We have already seen the power of collecting granular trace information for code coverage to analyze corpus and to drive fuzzing
- In this next section we will explore a few different binary instrumentation frameworks
- First we will implement our own code coverage plugins as an easy way to get our feet wet and then we will think about how to further improve our visibility into a program to extract information or detect vulnerabilities

# Dynamic Binary Instrumentation

- Binary translation is a robust program modification technique
  - JIT for hardware ISAs
- High level overview is straightforward
  - Copy code to cache for translation
  - Insert instructions to modify original binary
  - Link blocks into traces

# Dynamic Binary Instrumentation

- Performance comes from smart trace creation
  - Next Executing Tail
  - Traces begin at backedge or other trace exit

Pin

# Pin Architecture

- Features
  - Event Driven C++ API
  - Portable
    - x86/x64
    - Linux, Windows, OSX, Android
  - Trace granularity instrumentation
  - Begin at branch targets, end at indirect branch
  - Block/instruction level hooking supported
  - Higher level C++ API w/ helper routines
  - Closed source

# Pin Architecture

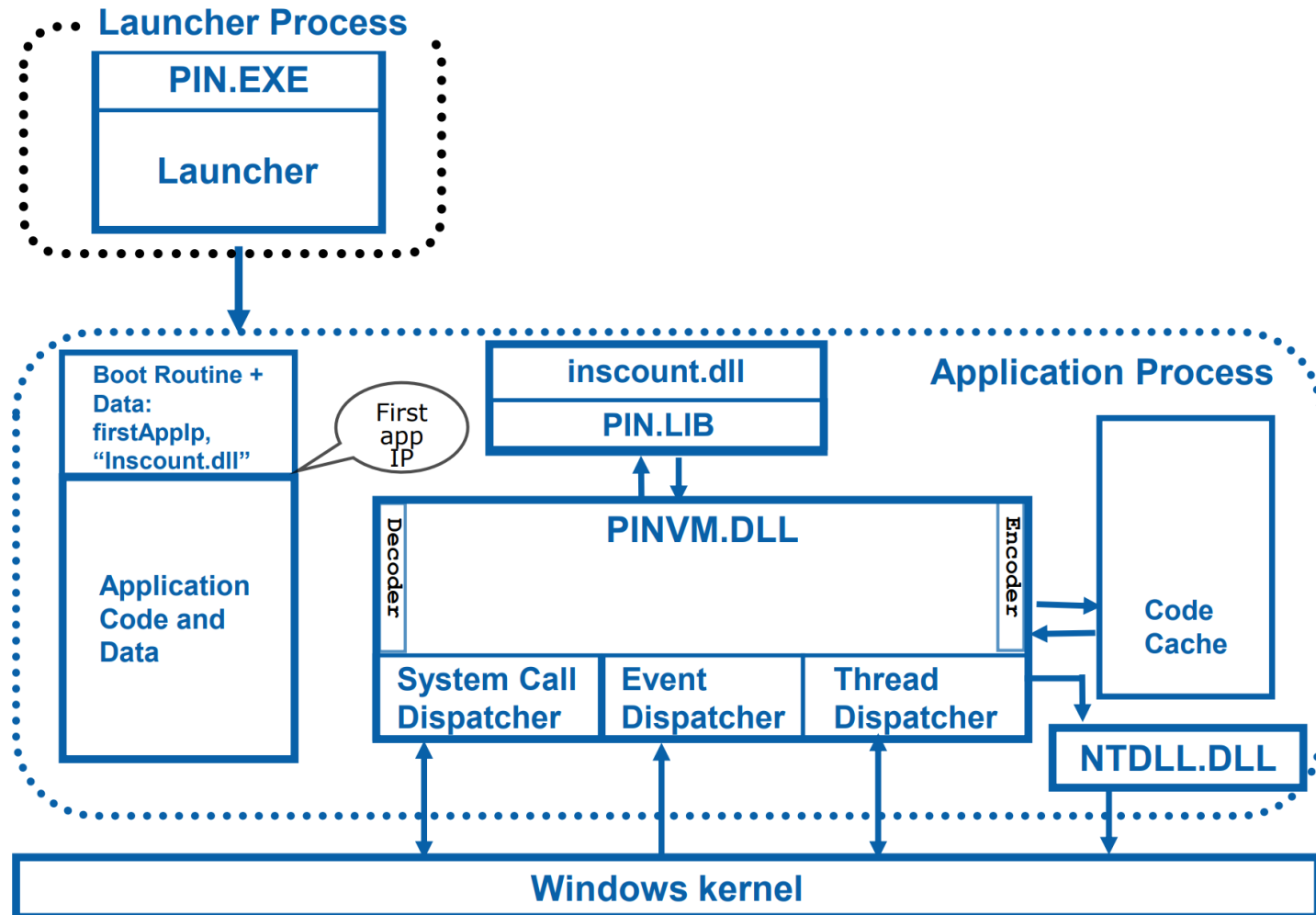
- Limitations
  - Delaying instrumentation until trace generation is slower
  - Limited inlining support
  - Less tuning options
  - Cannot observe blocks added to cache so 'hit trace' not possible
  - No mapping between BBL and traditional basic blocks
    - New BBLs are generated each time a trace is split
    - For jump tables, this means a new trace is generated each jump and only contains the jumps that come afterward
    - Extra instrumentation overhead at trace splitting

# Pin Architecture

- Pintools are plugins for Pin that implement callbacks for events to be notified when new code is discovered
- Instrumentation functions can be notified on:
  - Image Load
  - New Routine Discovery
  - New Basic Block Discovery
  - New Trace Generation
- Instrumentation functions can also iterate the instructions after notification
- Analysis functions are implemented by the pintool and injected as trampolines by the instrumentation functions using Pin API

# Pin Architecture

- Just-in-time binary instrumentation





# Pin API

- Pin makes it trivial to add trampolines at the beginning or end of a block or function or to wrap a system call or function call

# Code Coverage with Pin

```
int main(int argc, char *argv[])
{
 if(PIN_Init(argc, argv)){
 return Usage();
 }

 PIN_SetSyntaxIntel();
 TRACE_AddInstrumentFunction(Trace, 0);
 PIN_StartProgram();
}
```

# Code Coverage with Pin

```
VOID Trace(TRACE trace, VOID *v)
{
 for (BBL bbl = TRACE_BblHead(trace); BBL_Valid(bbl); bbl = BBL_Next(bbl))
 {
 //for (INS ins = BBL_InsHead(bbl); INS_Valid(ins); ins = INS_Next(ins))
 {
 BBL_InsertCall(bbl, IPOINT_BEGIN, (AFUNPTR)LogBranch,
 IARG_INST_PTR,
 IARG_END);
 }
 }
}
```

# Code Coverage with Pin

```
VOID LogBranch(ADDRINT cur_addr)
{
 std::cout << "\Block: 0x" << cur_addr << std::endl;
}
```

# Pin Exercise

- Code coverage with pin
- Navigate to `/vulndev/pin/source/tools/MyPinTool`
  - You can copy this directory and update the build files if you like
  - Or just develop on this project for now and worry about build issues later
- By default this tool counts basic blocks, modify it to output a list of blocks
- Add per-thread logging / look at Thread Local Storage use

# Pin Exercise

- Look at `/vulndev/pin/source/tools/SimpleExamples/calltrace.cpp`
- Merge in some hooks into your pintool for collecting indirect call targets in addition to the basic block trace

BREAK

DynamoRIO



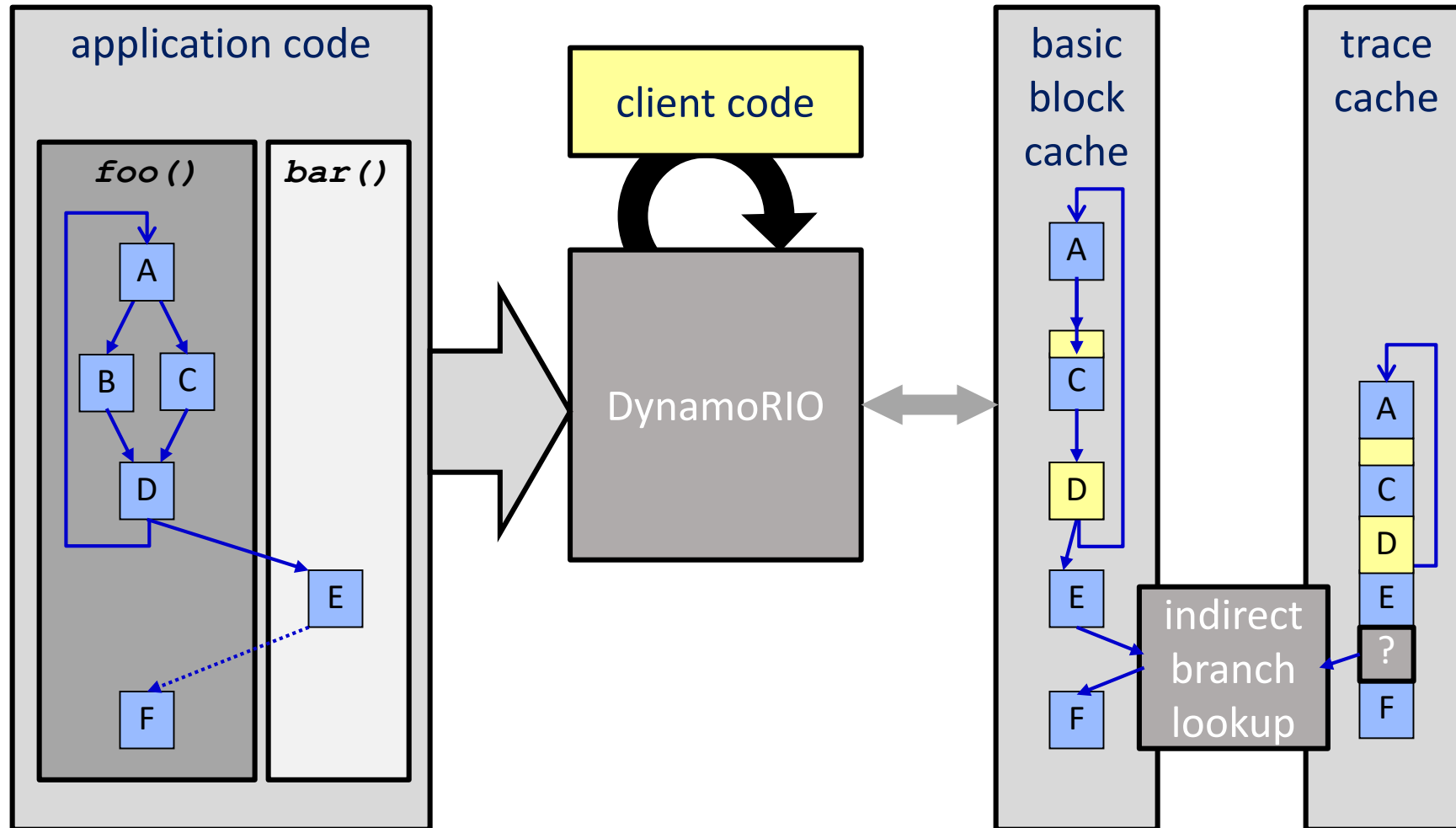
# DynamoRIO Overview

- Features
  - C API / BSD Licensed
  - Portable
    - x86/x64, ARM, AArch64
    - Linux, Windows, Android
- Block level instrumentation
- Blocks are directly copied into code cache
- Direct modification of IL possible

# DynamoRIO Overview

- Features
  - Much more flexible for block level instrumentation
  - Performance is a priority, Windows is a priority
  - Powerful tools like Dr Memory
  - Shadow memory, taint tracking
  - Twice as fast as Valgrind memcheck

# DynamoRIO Overview



# DynamoRIO Clients

- DynamoRIO
  - “*Interpret*” the application execution
  - API: provide a cross-platform interface for clients
    - Event callbacks
    - Utilities
- Client code
  - Extend DynamoRIO with customized operations
    - Event driven: operate on different events
    - Manipulate code stream
- DynamoRIO Extensions
  - Libraries that extend DynamoRIO API
    - *drmgr, drsyms, drwrap, ...*

# Events

- Application events
  - Runtime events occur during application execution
    - E.g. pre/post system calls
- DynamoRIO events
  - Code stream
    - Basic block/trace creation
  - Nudge
  - State restore
- Event callbacks registration
  - `dr_register_*_event`
  - `dr_unregister_*_event`

# Application Events

- Process start, stop, fork
- Thread creation and deletion
- Library load and unload
- Exception (Windows)
  - Client chooses whether to deliver or suppress
- Signal (Linux)
  - Client chooses whether to deliver, suppress, bypass, or redirect
- Pre- and post- system call
  - Platform-independent system call parameter access
  - Client can modify a system call
  - Invoke additional system call as the app

# DynamoRIO Events

- Code stream
  - Client has opportunity to inspect and potentially modify every single application instruction, immediately before it executes
    - Event happens at *transformation* time
    - Modifications or inserted code will operate at *execution* time
  - Entire application code stream
    - Basic block creation event: can modify the block
    - For comprehensive instrumentation tools
  - Or, focus on hot code only
    - Trace creation event: can modify the trace
    - Custom trace creation: can determine trace end condition
    - For optimization and profiling tools

# DynamoRIO Extensions

- DynamoRIO API is extended via libraries called Extensions
- Both static and shared supported
- Built and packaged with DynamoRIO
- Easy for a client to use with CMake
  - `use_DynamoRIO_extension(myclient extensionname)`



# DynamoRIO Extensions

- Current Extensions:
  - *drreg*: register stealing and allocating
  - *drsyms*: symbol table and debug information lookup
  - *drcontainers*: hashtable, vector, and table
  - *drmgr*: multi-instrumentation mediation
  - *drwrap*: function wrapping and replacing
  - *drutil*: memory tracing, string loop expansion
  - *drx*: multi-process management, misc utilities
  - *drsyscall*: system call monitoring: system call names, numbers, parameter types, memory references
  - *drdecode*: standalone IA32/AMD64/ARM/Thumb/AArch64 decoding/encoding library
  - *umbra*: shadow memory framework

# Code Coverage with DynamoRIO

```
// DynamoRIO initialization function
DR_EXPORT void
dr_init(client_id_t id)
{
 dr_register_exit_event(event_exit); //register exit callback
 dr_register_bb_event(event_basic_block); //register bb event callback
 // open log file
 fpCoverageLog = dr_open_file("blocks.log",
 DR_FILE_WRITE_OVERWRITE | DR_FILE_ALLOW_LARGE);
 DR_ASSERT(fpCoverageLog != INVALID_FILE);
}
```

# Code Coverage with DynamoRIO

```
// callback for each basic block executed
static dr_emit_flags_t
event_basic_block(void *drcontext, void *tag, instrlist_t *bb,
 bool for_trace, bool translating)
{
 char bbTag[20];
 dr_snprintf(bbTag , sizeof(bbTag), PFX"\n", tag);
 dr_write_file(fpCoverageLog, bbTag, sizeof(bbTag)-1);
 return DR_EMIT_DEFAULT;
}
```

# DynamoRIO Exercise

- Navigate to `/vulndev/dynamorio/samples`
  - The `wrap.c` sample shows the DynamoRIO way of wrapping APIs
- Make a backup copy of `wrap.c` and add simple code coverage output
- Optional: Take a look at `instrcalls.c` and add indirect call target output to your tool

# DynamoRIO Exercise

- We already have hooking implemented for malloc, lets add pageheap
  - Requested allocations are rounded up to page size and an additional page\_size is added to the request
  - The allocator returns the buffer to the interceptor function
  - The extra page added to the end is marked NOACCESS
  - The buffer is moved to the end of the rounded up page size
  - Any access beyond the end of the buffer will cause an exception

# DynamoRIO Exercise

- A useful addition to WinAFL will be locating functions that take our file path as an argument
- Now that we know how to hook calls, lets check to see if the function parameters match a user-specified string

# DynamoRIO API

Example: Adding a counter increment at each basic block

```
event_basic_block(void *drcontext, void *tag, instrlist_t *bb,
 bool for_trace, bool translating)
{
 instr_t *instr, *first = instrlist_first(bb);
 uint flags;
```

# DynamoRIO API

Example: Adding a counter increment at each basic block

```
if (!instr) dr_save_arith_flags(drcontext, bb, first, SPILL_SLOT_1);
for (instr = first; instr != NULL; instr = instr_get_next(instr))
{
 flags = instr_get_arith_flags(instr);
 /* OP_inc doesn't write CF but not worth distinguishing */
 if (TESTALL(EFLAGS_WRITE_6, flags) && !TESTANY(EFLAGS_READ_6,
 flags))
 break;
}
```



# DynamoRIO API

Example: Adding a counter increment at each basic block

```
instrlist_meta_preinsert(bb,
 (instr == NULL) ? first : instr,
 INSTR_CREATE_inc(drcontext,
 OPND_CREATE_ABSMEM((byte *)&global_count, OPSZ_4)));
if (instr == NULL)
 dr_restore_arith_flags(drcontext, bb, first, SPILL_SLOT_1);
return DR_EMIT_DEFAULT;
}
```

# DynamoRIO API

- A little more work but we've managed to check variable liveness, use a scratch register, and inline the counter increment we need to track code coverage
- This saves an expensive call out to a trampoline function

# DrCov

- We have already been using DrCov to prepare traces for Lighthouse
- DrCov takes advantages of all the performance features of DynamoRIO and should be a preferred tool for collecting code coverage on binary targets
- DrCov also comes with a script to convert to lcov format which is useful if you have source code available
- Additional tools ship in Lighthouse that add DrCov format to PIN and Frida

BREAK

DynInst

# DynInst

- Features
  - Static rewriting support
    - Dynamically linked binaries only
    - Eliminates issues with instruction cache misses common to DBT engines
  - Function level analysis
    - Tools must manually walk Dyninst provided CFG to instrument blocks
  - Modular C++ API / LGPL
- Observations
  - Fastest binary instrumentation out there
  - Development is slow
    - Patches we sent in for PE relocation support still not merged
  - Building Dyninst is NP-Hard
    - Use my Dockerfile on [github.com/talos-vulndev/afl-dyninst](https://github.com/talos-vulndev/afl-dyninst)

# DynInst

- Example

```
bool insertBBCallback(BPatch_binaryEdit * appBin, BPatch_function * curFunc,
 char *funcName, BPatch_function * instBBIncFunc,int *bbIndex)
{
 unsigned short randID;
 BPatch_flowGraph *appCFG = curFunc->getCFG ();
 BPatch_Set <BPatch_basicBlock *> allBlocks;
 BPatch_Set <BPatch_basicBlock *>::iterator iter;
 for (iter = allBlocks.begin (); iter != allBlocks.end (); iter++)
 {
 unsigned long address = (*iter)->getStartAddress ();

 randID = rand() % USHRT_MAX;
 BPatch_Vector <BPatch_snippet *> instArgs;
 BPatch_constExpr bbId (randID);
 instArgs.push_back (&bbId);
```

...

# DynInst

- Example

```
...
 BPatch_point *bbEntry = (*iter)->findEntryPoint();
 BPatch_funcCallExpr instIncExpr (*instBBIncFunc, instArgs);
 BPatchSnippetHandle *handle =
 appBin->insertSnippet (instIncExpr, *bbEntry, BPatch_callBefore,
 BPatch_lastSnippet);
 (*bbIndex)++;
}
return true;
}
```



# Tuning Binary Translation

- Only instrument indirect branches
- Delay instrumentation until input is seen
- Only instrument threads that access the data
- Move instrumentation logic to analysis routines
  - Some APIs provide IF-THEN-ELSE analysis with optimization
- Avoid trampolines
  - Be aware of code locality and instruction cache
  - Directly inline instructions, modify AST if possible
- Inject a fork server if repeatedly executing DBT
  - See our turbotrace tool

# TURBOTRACE

- TurboTrace is a tool for speeding up executions of a program that is run over each file in a directory (such as code coverage collection)
  - Fork in turbotrace LD\_PRELOAD library
  - Ptrace the forked child
  - Break on `_start`
  - Inject a call to the actual function that will be doing repeated `fork()`ing
  - Step over a call
  - Repair the `_start` and resume execution

# TURBOTRACE

- TurboTrace performance, 100 iterations
  - 20 – 50% speed increase

First test (without pintool, just instrumentation):

Pin            without pintool on test\_png : 55.03 seconds

Turbotrace without pintool on test\_png : 37.24 seconds

Second test (bblocks pintool):

Pin            bblocks pintool on test\_png : 72.62 seconds

Turbotrace bblocks pintool on test\_png : 51.07 seconds

Second test (calltrace pintool):

Pin            calltrace pintool on test\_png : 106.19 seconds

Turbotrace calltrace pintool on test\_png : 85.24 seconds

# TurboTrace Exercise

- Use turbotrace to speed up your Pin and DynamoRIO tools